

Hình học tính toán (Computational Geometry)

PHỤ THUỘC B/07 cho ngăn xếp trong thuật toán bao lồi và cho hàng đợi ưu tiên trong quét đường; C/14 cho bài cặp điểm gần nhất; D/21 cho tinh thần “cạm bẫy nằm ở số học”.

TÓM TẮT

Hình học tính toán có một đặc điểm khác hẳn mọi chương trước: thuật toán thường đơn giản, nhưng *số dấu phẩy động* có thể phá hủy tính đúng theo những cách rất khó truy. Vì vậy chương bắt đầu từ vị từ định hướng — viên gạch mà gần như mọi thuật toán hình học đứng trên — và từ câu hỏi làm sao tính nó cho đáng tin. Sau đó là bao lồi, quét đường, và các bài kinh điển: giao đoạn thẳng, điểm trong đa giác, cặp điểm gần nhất, cặp điểm xa nhất. Nếu chỉ nhớ một điều: hãy làm việc với số nguyên bất cứ khi nào có thể, và khi buộc phải dùng số thực, hãy hiểu rằng “bằng nhau” không còn là một khái niệm dùng được.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tích có hướng (cross product)** — với hai vector trong mặt phẳng, $u \times v = u_x v_y - u_y v_x$; dấu của nó cho biết v nằm bên trái hay bên phải u .
- **vị từ định hướng** — hàm trả lời “ba điểm này quay trái, quay phải, hay thẳng hàng” — tính bằng dấu của tích có hướng. Nền của gần như mọi thuật toán hình học.
- **bao lồi** — đa giác lồi nhỏ nhất chứa tất cả các điểm đã cho; hình dung là dây thun bọc quanh các đỉnh.
- **quét đường** — kỹ thuật cho một đường thẳng tưởng tượng quét qua mặt phẳng, duy trì một cấu trúc dữ liệu về các đối tượng đang cắt nó.
- **suy biến (degenerate case)** — cấu hình đặc biệt — ba điểm thẳng hàng, hai điểm trùng nhau, đoạn thẳng chồng nhau — nơi phần lớn lỗi hình học nằm.
- **vị từ chính xác** — cách tính vị từ đảm bảo đúng dấu, thường bằng số nguyên hoặc bằng số học nhiều độ chính xác.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “ba điểm có thẳng hàng không” lại là một câu hỏi khó khi toạ độ là số thực?
- Q2. Bao lồi có thể tính bằng nhiều thuật toán khác nhau. Vì sao cách sắp xếp rồi quét lại phổ biến hơn cả?
- Q3. Kỹ thuật quét đường biến bài toán hai chiều thành bài toán một chiều. Cơ chế nào cho phép làm điều đó?
- Q4. Vì sao một lỗi làm tròn nhỏ trong hình học lại có thể gây ra kết quả hoàn toàn vô nghĩa, chứ không chỉ sai một chút?
- Q5. Khi nào nên dùng số nguyên cho bài hình học, và làm sao tránh tràn khi toạ độ tới 10^9 ?

Có một hiện tượng đặc trưng khi làm bài hình học: chương trình chạy đúng trên mọi ví dụ bạn tự nghĩ ra, rồi sai trên bộ test thật vì có ba điểm thẳng hàng, hoặc hai điểm trùng nhau, hoặc một đoạn thẳng thẳng đứng. Hình học là miền mà *trường hợp suy biến* chiếm phần lớn số lỗi, và điều

đó có lý do sâu xa: thuật toán hình học thường dựa trên các quyết định nhị phân — bên trái hay bên phải, trong hay ngoài — và ở đúng ranh giới giữa hai lựa chọn, mọi thứ trở nên mong manh.

Cộng thêm một tầng khó nữa: nếu toạ độ là số thực, các quyết định đó được tính bằng số dấu phẩy động, vốn có sai số. Một quyết định sai không chỉ làm kết quả lệch một chút mà có thể làm thuật toán rơi vào trạng thái *không nhất quán* — ví dụ tin rằng điểm A ở bên trái điểm B và đồng thời B ở bên trái A — và từ đó cho ra kết quả hoàn toàn vô nghĩa hoặc treo vòng lặp.

Vì vậy chương này ưu tiên hai thứ: các vị từ nền tảng và cách tính chúng cho đáng tin, rồi mới tới thuật toán.

1.1 Vị từ định hướng: viên gạch của mọi thứ

Cho ba điểm A, B, C , câu hỏi cơ bản nhất là: đi từ A tới B rồi tới C , ta rẽ trái, rẽ phải, hay đi thẳng? Câu trả lời nằm ở dấu của tích có hướng

$$(B - A) \times (C - A) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

Dương là rẽ trái, âm là rẽ phải, bằng 0 là thẳng hàng.

Điều đáng nói là gần như mọi thuật toán hình học trong chương này chỉ dùng vị từ đó và một hai vị từ họ hàng. Kiểm tra hai đoạn thẳng có cắt nhau, kiểm tra điểm nằm trong đa giác lồi, xây bao lồi, sắp xếp theo góc — tất cả đều là các cách kết hợp vị từ định hướng. Hệ quả thực hành rất quan trọng: nếu bạn tính vị từ này *đúng dấu* thì phần lớn thuật toán sẽ chạy đúng; nếu tính sai dấu ở các trường hợp gần suy biến thì mọi thứ sụp.

Từ đó có một quy tắc đáng thành phần xa: *làm việc với số nguyên bất cứ khi nào có thể*. Tích có hướng của các toạ độ nguyên là số nguyên, nên dấu của nó tính được chính xác tuyệt đối — miễn là không tràn. Với toạ độ tới 10^9 , tích có hướng có thể tới $4 \cdot 10^{18}$, sát giới hạn số nguyên 64 bit có dấu; nếu toạ độ lớn hơn hoặc nếu bạn nhân thêm lần nữa thì cần kiểu 128 bit. Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

CẠM BẮY

Khi buộc phải dùng số thực, đừng bao giờ viết `if (a == b)`. Thay bằng so sánh với một ngưỡng: `if (fabs(a-b) < eps)`. Nhưng phải hiểu rằng đây là một sự thoả hiệp chứ không phải lời giải: quan hệ “gần bằng nhau” *không có tính bắc cầu*, nên nếu bạn dùng nó làm quan hệ so sánh trong một hàm sắp xếp, kết quả có thể là không xác định — kể cả sắp chương trình trong một số thư viện. Chọn ε cũng không có công thức chung: quá nhỏ thì không bắt được trường hợp suy biến, quá lớn thì gộp nhầm những điểm thật sự khác nhau.

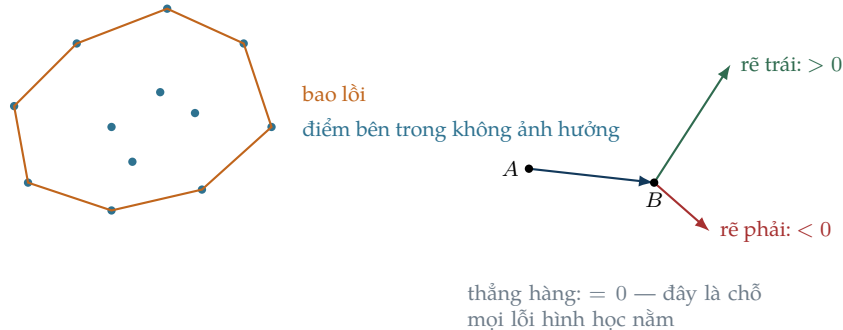
1.2 Bao lồi: thuật toán đầu tiên nên thuộc

Bài toán: cho n điểm, tìm đa giác lồi nhỏ nhất chứa tất cả. Cách hình dung tốt nhất là một sợi dây thun bọc quanh các cây đinh đóng trên bảng.

Cách phổ biến nhất — và cũng dễ cài đúng nhất — là *quét đơn điệu*: sắp các điểm theo hoành độ (rồi tung độ), sau đó xây nửa trên và nửa dưới của bao bằng cách quét một lượt và duy trì một ngăn xếp, loại bỏ mọi điểm làm ta “rẽ sai chiều”. Chi phí $O(n \log n)$, chi phối bởi bước sắp xếp.

Cấu trúc của thuật toán đáng để ý: nó là một *ngăn xếp đơn điệu* (B/07) áp lên tiêu chí hình học thay vì tiêu chí số học. Mỗi điểm vào ngăn xếp một lần và ra tối đa một lần, nên phần quét là $O(n)$ — đúng lập luận khẩu hao quen thuộc.

Bao lồi hữu ích hơn về ngoài của nó vì nhiều bài quy về nó. Cặp điểm *xa nhất* trong tập luôn nằm trên bao lồi, và tìm được bằng kỹ thuật hai con trỏ quay quanh bao. Diện tích nhỏ nhất của hình bao, hình chữ nhật nhỏ nhất bao quanh, kiểm tra một điểm có nằm trong bao — tất cả đều dựa trên bao lồi. Trong tối ưu hoá, bao lồi cũng chính là cấu trúc đẳng sau kỹ thuật tăng tốc quy hoạch động ở C/16: tập các đường thẳng cần lấy cực trị tạo thành một bao lồi.



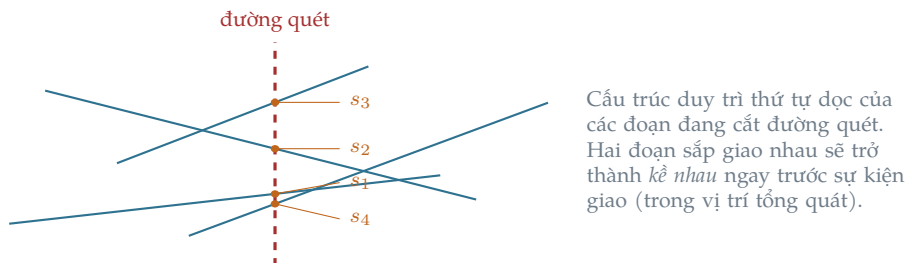
Hình 1.1. Bao lồi và vị từ định hướng. Bên phải là viên gạch: dấu của tích có hướng quyết định mọi thứ. Trường hợp bằng 0 — ba điểm thẳng hàng — là nơi phần lớn lỗi xảy ra, vì đó là ranh giới giữa hai nhánh xử lý, và cũng là nơi sai số dấu phẩy động có thể lật ngược quyết định.

1.3 Quét đường: biến hai chiều thành một chiều

Nhiều bài hình học có dạng “xét mọi cặp đối tượng” và cho $O(n^2)$ một cách hiển nhiên. Kỹ thuật quét đường thường đưa xuống $O(n \log n)$ bằng một ý tưởng chung: cho một đường thẳng đứng quét từ trái sang phải, và duy trì một cấu trúc dữ liệu chứa các đối tượng *đang cắt* đường quét đó, sắp theo tung độ.

Cơ chế làm nó hiệu quả — và đây là câu trả lời cho câu hỏi mở đầu thứ ba — là: tại mỗi thời điểm, ta chỉ cần quan tâm tới quan hệ giữa các đối tượng *kề nhau* trong thứ tự dọc. Hai đoạn thẳng chỉ có thể cắt nhau nếu tại một lúc nào đó chúng trở thành *kề nhau* trên đường quét. Nhờ đó, thay vì xét n^2 cặp, ta chỉ xử lý các sự kiện: một đối tượng bắt đầu, một đối tượng kết thúc, hai đối tượng đổi chỗ. Số sự kiện là $O(n)$ cộng số giao điểm, và mỗi sự kiện tốn $O(\log n)$.

Ba bài kinh điển giải bằng quét đường: tìm mọi giao điểm của n đoạn thẳng; hợp diện tích của n hình chữ nhật; và cặp điểm gần nhất — bài mà chương C/14 giải bằng chia để trị, còn quét đường cho một lời giải khác cũng $O(n \log n)$ nhưng dễ cài hơn với một tập hợp có thứ tự.



Hình 1.2. Quét đường. Bài toán hai chiều trở thành bài toán duy trì một tập có thứ tự trong một chiều, tức là quay về đúng công cụ của B/09. Số sự kiện phải xử lý là $O(n)$ cộng số giao điểm, thay vì $O(n^2)$ cặp.

1.4 Bốn bài nền và các bẫy của chúng

Điểm có nằm trong đa giác không. Với đa giác lồi: kiểm bằng vị từ định hướng cho mọi cạnh, hoặc bằng tìm kiếm nhị phân trên góc cho $O(\log n)$. Với đa giác bất kỳ: bắn một tia và đếm số lần cắt biên — lẻ là trong, chẵn là ngoài. Bẫy: tia đi qua đúng một đỉnh sẽ bị đếm hai lần hoặc không lần nào; xử lý điểm trên biên riêng, rồi chỉ xét cạnh có $(y_i > y_p) \neq (y_j > y_p)$ và giao tia về phía phải. Quy ước nửa mở này đếm một đỉnh đúng một lần; yêu cầu hai đầu nghiêm ngặt ở trên/dưới sẽ bỏ sót đỉnh nằm trên tia.

Diện tích đa giác. Công thức dây giày cho diện tích có dấu bằng tổng các tích có hướng chia đôi. Dấu cho biết chiều duyệt, và đó là thông tin hữu ích chứ không phải phiền toái. Bẫy: quên chia 2, và tràn số khi tọa độ lớn.

Hai đoạn thẳng có cắt nhau không. Trường hợp thường thì bốn lần gọi vị từ định hướng là đủ. Bẫy nằm ở trường hợp suy biến: bốn điểm thẳng hàng, khi đó phải kiểm thêm xem hai đoạn có chồng lên nhau không bằng cách so hình chiếu lên hai trục.

Sắp xếp theo góc. Rất nhiều bài cần sắp các điểm quanh một tâm theo góc. Bẫy lớn nhất là dùng hàm arctan — vừa chậm vừa có sai số. Cách đúng là chia các điểm thành nửa trên và nửa dưới rồi so bằng tích có hướng, hoàn toàn bằng số nguyên.

1.5 Vì sao sai số nhỏ lại gây hậu quả lớn

Đây là mục lý giải cho câu hỏi mở đầu thứ tư, và nó là điều phân biệt hình học tính toán với các miền khác.

Trong hầu hết tính toán số, một sai số nhỏ cho một kết quả sai nhỏ — bạn tính ra 3,0001 thay vì 3,0000 và không sao. Trong hình học, các đại lượng số được dùng để đưa ra *quyết định rời rạc*: điểm này nằm trong hay ngoài, ba điểm này thẳng hàng hay không. Một sai số nhỏ ở gần ranh giới làm lật ngược quyết định, và từ một quyết định sai, cấu trúc dữ liệu của thuật toán rơi vào trạng thái không tương ứng với bất kỳ cấu hình hình học hợp lệ nào.

Hậu quả không phải là “kết quả lệch một chút” mà là các sự cố có tính chất khác hẳn: bao lỗi tự cắt chính nó, thuật toán quét đường lặp vô hạn vì thứ tự trên đường quét mâu thuẫn, hoặc chương trình sập vì một hàm sắp xếp nhận được quan hệ so sánh không nhất quán.

Có ba cách đối phó, xếp theo thứ tự nên thử. *Một*: dùng số nguyên — luôn ưu tiên, và trong lập trình thi đấu thì đề bài thường cho tọa độ nguyên chính là vì lý do này. *Hai*: dùng số học chính xác cho riêng các vị từ, tức là chỉ trả giá ở những chỗ quyết định. *Ba*: nếu chấp nhận dung sai, phải định nghĩa rõ bài toán xấp xỉ và quy tắc xử lý suy biến. Phép quay bảo toàn tính thẳng hàng nên không thể loại bỏ ba điểm thẳng hàng; nhiều tọa độ thì thay đổi dữ liệu và có thể thay đổi đáp án. Khi cần dấu chính xác, dùng vị từ thích nghi với kiểm soát sai số[1].

1.6 Vị từ thích nghi: trả giá chính xác đúng chỗ cần

Cách thứ hai ở mục trên — “dùng số học chính xác cho riêng các vị từ” — nghe như một lời khuyên chung chung, nhưng nó có một cài đặt cụ thể đã trở thành chuẩn công nghiệp, và cơ chế của nó đáng hiểu vì nó là một mẫu thiết kế dùng lại được.

Vấn đề với số học chính xác thuần túy là giá: nếu mọi phép định hướng đều tính bằng số hữu tỉ chính xác, chương trình chậm đi hàng chục lần, trong khi hầu hết các lần gọi đều ở xa ranh giới tới mức số thực thường đã cho dấu đúng. Ý tưởng của Shewchuk[1] là *thích nghi*: tính vị từ theo nhiều tầng, tầng đầu rẻ nhất, và mỗi tầng đi kèm một *chặn sai số* tính được ngay trong lúc chạy.

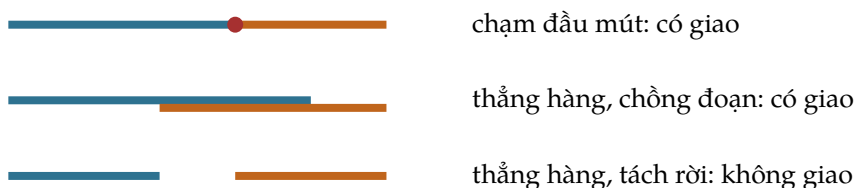
Nếu giá trị ước lượng lớn hơn chặn sai số của nó thì dấu chắc chắn đúng và ta dừng — đây là trường hợp của gần như mọi dữ liệu thật. Chỉ khi giá trị rơi vào vùng không phân biệt được với 0, thuật toán mới leo lên tầng chính xác hơn, và chỉ ở những cấu hình gần suy biến ta mới trả giá đầy đủ.

Kết quả là một hàm vừa *luôn* cho dấu đúng vừa nhanh gần bằng số thực trên dữ liệu bình thường — và đó là lý do các thư viện tam giác phân, lưới và bản đồ đều dựa trên loại vị từ này. Mẫu tư duy chung, đáng mang sang chỗ khác: *tính rẻ trước kèm một chứng chỉ về sai số, và chỉ nâng cấp chi phí khi chứng chỉ không đủ để kết luận*. Cùng khuôn ấy xuất hiện ở bộ lọc trước một phép kiểm đất tiền (B/08) và ở việc cắt nhánh trong quay lui (C/13).

Một hệ quả thực hành đi kèm, thường bị bỏ qua: nếu bạn so sánh bằng ngưỡng ε , quan hệ “bằng nhau” sinh ra *không bắc cầu* — có thể $a \approx b$ và $b \approx c$ mà $a \not\approx c$. Đưa một quan hệ so sánh như vậy cho hàm sắp xếp của thư viện là hành vi không xác định, và triệu chứng của nó là chương trình sập ở một chỗ chẳng liên quan gì tới hình học. Đây chính là loại sự cố mà mục trước gọi là “hậu quả khác hẳn”, và cách phòng duy nhất chắc chắn là đừng để ε lọt vào quan hệ thứ tự.

1.7 Bộ hình suy biến phải thử trước dữ liệu ngẫu nhiên

Hình 1.3 gom ba cấu hình mà phép kiểm chỉ dựa vào dấu nghiêm ngặt thường bỏ sót. Trước khi viết code, quyết định “giao” có tính chạm đầu mút không; hình dưới dùng đoạn đóng, nên hai trường hợp đầu đều có giao.



Hình 1.3. Ba ca kiểm thử cho giao hai đoạn đóng. Hàng giữa dịch nét màu cam xuống một chút để nhìn thấy phần chồng; dữ liệu toán học vẫn có cùng tung độ. Ba cấu hình đều có định hướng bằng 0, nên phải kiểm thêm hình chiếu hoặc điểm nằm trên đoạn.

Kinh nghiệm số học: ép sang kiểu rộng *trước* phép trừ và nhân, không đợi tới khi gán kết quả. Chẳng hạn `wide(B.x) - wide(A.x)` bảo vệ phép trừ; ép kiểu sau `B.x - A.x` không cứu được phép trừ đã tràn. Với dấu của hai định hướng, so dấu trực tiếp thay vì nhân chúng, vì tích hai định thức có thể tràn dù từng định thức còn vừa kiểu số. Thêm các ca điểm trùng nhau, đoạn dài 0 và tọa độ ở hai cực của miền cho phép.

1.8 Gốc rễ: một luận án năm 1978 và cuộc khủng hoảng về độ tin cậy

Hình học tính toán là ngành trẻ so với các chương khác: nó thường được tính là bắt đầu từ luận án tiến sĩ của Michael Shamos năm 1978, quyển đầu tiên coi “các bài toán hình học” như một lĩnh vực có phương pháp riêng thay vì một tập hợp bài lẻ. Trước đó, các thuật toán hình học tồn tại rải rác — thuật toán bao lồi của Graham ra đời năm 1972, và động cơ của nó rất thực tế: xử lý dữ liệu thống kê, nơi bao lồi được dùng để tìm các điểm cực biên.

Kỹ thuật quét đường được hệ thống hoá cuối thập niên 1970, với thuật toán tìm giao điểm các đoạn thẳng của Bentley và Ottmann năm 1979 — cùng Bentley của *Programming Pearls* ở chương

A/02. Cùng thời kỳ, các cấu trúc chia không gian như cây k -d và cây khoảng ra đời để trả lời truy vấn hình học, và chúng chính là các cây tìm kiếm được tăng cường mà B/09 đã nói.

Nhưng câu chuyện đáng nhớ nhất của ngành này là câu chuyện về *độ tin cậy số học*, và nó đáng kể vì nó là một trường hợp hiếm hoi mà cả một cộng đồng phải thừa nhận rằng lý thuyết đã bỏ sót một điều căn bản. Suốt thập niên 1980, các thuật toán hình học được công bố với chứng minh đầy đủ, nhưng khi cài đặt bằng số thực thì chúng sập hoặc cho kết quả vô nghĩa trên dữ liệu thật. Vấn đề không phải lỗi lập trình mà là sự lệch pha giữa mô hình — “số thực chính xác vô hạn” — và thực tế — “số dấu phẩy động 64 bit”.

Phản ứng của ngành trong thập niên 1990 là xây dựng *tính toán vị từ chính xác*: các kỹ thuật tính dấu của biểu thức hình học một cách đảm bảo, thường bằng cách ước lượng trước với số dấu phẩy động rồi chỉ chuyển sang số học chính xác khi kết quả gần 0. Ý tưởng “chỉ trả giá cho độ chính xác ở nơi thật sự cần” này về sau thành chuẩn, và nó được đóng gói trong các thư viện hình học công nghiệp. Bài học cho người học: khi ai đó nói “thuật toán này đã được chứng minh”, hãy hỏi thêm “chứng minh trong mô hình số nào”.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1972 — Graham	Tìm các điểm cực biên trong dữ liệu thống kê.	Thuật toán bao lồi bằng sắp xếp theo góc và quét bằng ngăn xếp.
1978 — Shamos	Các bài toán hình học được giải lẻ tẻ, không có phương pháp chung.	Luận án khai sinh hình học tính toán như một lĩnh vực với các kỹ thuật chuẩn.
1979 — Bentley & Ottmann	Tìm mọi giao điểm của n đoạn thẳng nhanh hơn $O(n^2)$.	Quét đường: biến bài hai chiều thành chuỗi sự kiện một chiều.
Thập niên 1980 — cấu trúc truy vấn hình học	Trả lời nhiều truy vấn trên cùng tập điểm.	Cây k -d, cây khoảng, cây phân đoạn — đều là cây tìm kiếm được tăng cường (B/09, B/10).
Thập niên 1990 — khủng hoảng độ tin cậy	Thuật toán đã chứng minh vẫn sập khi cài bằng số thực.	Nhận ra khoảng cách giữa mô hình số thực lý tưởng và số dấu phẩy động; ra đời kỹ thuật vị từ chính xác thích ứng.
Thập niên 1990–2000 — thư viện hình học	Mỗi nhóm tự cài lại và tự mắc lại cùng các lỗi.	Các thư viện đóng gói vị từ chính xác và xử lý suy biến, cho phép dùng hình học tính toán trong sản phẩm thật.

1.9 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Đồ hoạ máy tính	Cắt đa giác, kiểm tra va chạm, dò tia	Cùng vị từ nền tảng. Khác về ưu tiên: đồ hoạ chấp nhận sai số nhỏ để đổi lấy tốc độ khung hình, còn hình học tính toán thường cần tính đúng tuyệt đối — hai cộng đồng cùng thuật toán, khác tiêu chuẩn.
Hệ thông tin địa lý	Chồng bản đồ, truy vấn vùng, tìm điểm gần nhất	Quét đường và cây phân vùng không gian là công cụ nền. Ở đây dữ liệu suy biến rất phổ biến (đường biên chung, điểm trùng), nên xử lý suy biến quan trọng hơn cả tốc độ.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Robot và SLAM	Kiểm tra va chạm, bản đồ không gian tự do, khớp đám mây điểm	Bao lồi dùng để mô tả vùng chiếm chỗ của vật thể; cây k -d dùng để tìm điểm gần nhất trong bước khớp đám mây. Điểm khác: dữ liệu cảm biến có nhiều nên các vị từ chính xác ít giá trị hơn — cái cần là bền vững với nhiễu, không phải chính xác tuyệt đối.
Thiết kế và chế tạo	CAD, sinh lưới, in ba chiều	Sinh lưới tam giác dựa trên tam giác phân Delaunay; các lỗi suy biến ở đây gây hậu quả vật lý thật, nên ngành này là nơi vị từ chính xác được coi trọng nhất.
Tối ưu hoá	Bao lồi trong quy hoạch tuyến tính	Miền khả thi của quy hoạch tuyến tính là một đa diện lồi, và thuật toán đơn hình đi trên các đỉnh của nó. Bao lồi trong chương này là bản hai chiều của cùng đối tượng.
Quy hoạch động	Kỹ thuật bao lồi để tăng tốc chuyển trạng thái	Tập các đường thẳng cần lấy cực trị tạo thành một bao lồi; kỹ thuật ở C/16 chính là bao lồi động. Một liên hệ đẹp giữa hình học và tối ưu tổ hợp.
Học máy	Máy vector hỗ trợ, phân tách tuyến tính	Việc tìm siêu phẳng phân tách hai lớp có liên hệ trực tiếp với khoảng cách giữa hai bao lồi của hai lớp — hình học lồi là ngôn ngữ chung.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì với số thực, “bằng 0” gần như không bao giờ xảy ra đúng nghĩa: kết quả tính tích có hướng của ba điểm thẳng hàng có thể ra 10^{-16} thay vì 0 do sai số làm tròn. Bạn buộc phải so với một ngưỡng, mà ngưỡng đó không có giá trị đúng phổ quát — nó phụ thuộc độ lớn của toạ độ và số phép toán đã thực hiện. Tệ hơn, quan hệ “gần bằng” không bắc cầu, nên nó không dùng được ở những nơi cần một quan hệ so sánh nhất quán.
- A2. Vì nó dễ cài đúng và ít trường hợp riêng nhất. Sau khi sắp xếp, thuật toán chỉ là một vòng quét với ngăn xếp và một phép kiểm tra vị từ định hướng — ít chỗ để sai. Các thuật toán khác như gói quà có thể nhanh hơn khi bao có ít đỉnh, nhưng chúng nhạy cảm hơn với các điểm thẳng hàng và trùng nhau. Trong hình học, tiêu chí “ít trường hợp riêng” đáng giá hơn một chút khác biệt về tiệm cận.
- A3. Cơ chế là: tại mỗi thời điểm, chỉ các đối tượng *kề nhau* theo thứ tự dọc trên đường quét mới có thể tương tác. Hai đoạn thẳng chỉ cắt nhau nếu có lúc chúng *kề nhau* trong thứ tự đó. Nhờ vậy, thay vì n^2 cặp, ta chỉ cần xử lý các sự kiện làm thay đổi quan hệ *kề*: một đối tượng vào, một đối tượng ra, hai đối tượng đổi chỗ. Bài toán hai chiều trở thành bài toán duy trì một tập có thứ tự trong một chiều, tức là quay về cấu trúc dữ liệu của B/09.
- A4. Vì các đại lượng số ở đây được dùng để ra *quyết định rời rạc*, không phải để báo cáo giá trị. Sai số nhỏ ở gần ranh giới lật ngược quyết định, và một quyết định sai đưa cấu trúc dữ liệu vào trạng thái không tương ứng với cấu hình hình học hợp lệ nào — ví dụ thứ tự trên đường quét mâu thuẫn với chính nó. Từ đó hậu quả không phải sai số nhỏ mà là bao lồi tự cắt, vòng lặp không dừng, hoặc chương trình sập.
- A5. Luôn nên dùng số nguyên khi đề cho toạ độ nguyên — và đề thi thường cố ý cho như vậy chính vì lý do này. Với toạ độ tới 10^9 , tích có hướng có thể tới $4 \cdot 10^{18}$, vẫn nằm trong số nguyên 64 bit có dấu nhưng rất sát; nếu công thức của bạn nhân thêm một lần nữa hoặc cộng nhiều số hạng thì phải chuyển sang kiểu 128 bit. Mẹo giảm rủi ro: tính tiến toàn bộ tập điểm về gần gốc toạ độ trước khi tính, vì tích có hướng chỉ phụ thuộc hiệu các toạ độ.

NÓI LẠI THẬT ĐƠN GIẢN

Hình học trên máy tính có một điều lạ: bài toán thì trực quan, nhưng cái làm hỏng chương trình lại không nằm ở chỗ bạn nghĩ.

Gần như mọi thuật toán trong chương này đứng trên một câu hỏi duy nhất: đi từ điểm A tới B rồi tới C thì bạn rẽ trái hay rẽ phải? Có một công thức nhỏ trả lời câu đó bằng một phép nhân và một

phép trừ. Bao lỗi, kiểm tra hai đoạn cắt nhau, kiểm tra điểm nằm trong hình — tất cả chỉ là cách xâu chuỗi câu hỏi ấy lại.

Rắc rối bắt đầu khi ba điểm nằm gần như thẳng hàng. Lúc đó câu trả lời đúng là “không rẽ bên nào cả”, nhưng máy tính dùng số thực có sai số, nên nó có thể nói “rẽ trái một chút” hoặc “rẽ phải một chút” tùy vào bụi bặm của phép làm tròn. Và vì thuật toán dựa trên câu trả lời đó để quyết định, một câu trả lời sai không làm kết quả lệch chút xíu — nó làm chương trình tin vào một thế giới không tồn tại, kiểu như tin rằng A ở bên trái B và B cũng ở bên trái A. Từ đó thì bất cứ chuyện gì cũng có thể xảy ra: hình vẽ tự cắt chính nó, vòng lặp chạy mãi, hoặc chương trình sập.

Cách phòng đơn giản nhất và hiệu quả nhất: nếu toạ độ là số nguyên thì hãy giữ nguyên số nguyên từ đầu tới cuối, đừng đổi sang số thực. Phép nhân và trừ trên số nguyên cho kết quả chính xác tuyệt đối, nên câu hỏi “rẽ bên nào” luôn được trả lời đúng. Chỉ cần canh chừng một chuyện: số nguyên cũng có giới hạn, và với toạ độ cỡ một tỉ thì tích của chúng đã sát trần.

Một câu: Mọi thuật toán hình học đứng trên một câu hỏi “rẽ trái hay rẽ phải” — và tính đúng của cả chương trình phụ thuộc vào việc trả lời câu đó cho chính xác.

Chỗ không rút gọn được: Không có giá trị ε đúng phổ quát; việc chọn nó luôn là thỏa hiệp phụ thuộc dữ liệu cụ thể.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vị từ định hướng	Dấu của $(B - A) \times (C - A)$: dương là rẽ trái, âm là rẽ phải, 0 là thẳng hàng. Nền của gần như mọi thuật toán hình học.
••• Quy tắc số học	Dùng số nguyên bất cứ khi nào có thể; toạ độ tới 10^9 cho tích có hướng tới $4 \cdot 10^{18}$ — sát trần 64 bit.
••• Vì sao sai số nhỏ gây hậu quả lớn	Số được dùng để ra quyết định rồi rắc; quyết định sai đưa cấu trúc vào trạng thái không tương ứng cấu hình hợp lệ nào $\Rightarrow$ tự cắt, lặp vô hạn, sập.
••• Bao lỗi bằng quét đơn điệu	Sắp theo hoành độ rồi quét hai nửa với ngăn xếp đơn điệu; $O(n \log n)$, chi phối bởi sắp xếp; ít trường hợp riêng nhất.
••• Cơ chế của quét đường	Chỉ các đối tượng <i>kê nhau</i> theo thứ tự dọc mới tương tác; xử lý sự kiện vào/ra/đổi chỗ thay vì mọi cặp.
•• So sánh số thực	Không bao giờ dùng $==$; dùng ngưỡng ε nhưng nhớ rằng “gần bằng” <i>không bắc cầu</i> nên không dùng làm quan hệ sắp xếp.
•• Điểm trong đa giác	Lỗi: kiểm vị từ mọi cạnh, hoặc nhị phân theo góc $O(\log n)$. Bất kỳ: bắn tia đếm số lần cắt, lẻ là trong; bẫy là tia đi qua đỉnh.
•• Diện tích đa giác	Công thức dây giày: tổng tích có hướng chia đôi; dấu cho biết chiều duyệt.
•• Sắp xếp theo góc	Chia nửa trên / nửa dưới rồi so bằng tích có hướng — hoàn toàn số nguyên; đừng dùng arctan.
•• Cặp điểm xa nhất	Luôn nằm trên bao lỗi; tìm bằng hai con trỏ quay quanh bao sau khi có bao.
•• Ba cách đối phó sai số	Số nguyên (ưu tiên); vị từ chính xác thích ứng (chỉ trả giá ở chỗ quyết định); hoặc định nghĩa dung sai và xử lý suy biến rõ ràng. Phép quay không loại tính thẳng hàng.
• Liên hệ với quy hoạch động	Kỹ thuật bao lỗi tăng tốc chuyển trạng thái ($C/16$) chính là bao lỗi động trên tập đường thẳng.
•• Vị từ thích nghi	Tính rẽ trước kèm một chặn sai số tính ngay lúc chạy; chỉ leo lên tầng chính xác hơn khi giá trị không phân biệt được với 0. Kèm theo: so sánh bằng ε không bắc cầu, đừng đưa vào hàm sắp xếp.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên lưu điểm bằng kiểu gì trong bài hình học?

| Nếu đề cho toạ độ nguyên, hãy dùng số nguyên 64 bit và giữ nguyên suốt — kể cả khi công thức trung gian trông có vẻ cần chia. Rất nhiều bài tránh được phép chia bằng cách nhân chéo thay vì so hai phân số. Chỉ chuyển sang số thực ở bước cuối cùng khi phải in ra kết quả có phần thập phân. Nếu đề cho toạ độ thực, hãy cân nhắc nhân tất cả lên một hệ số rồi làm tròn — nhiều bài cho toạ độ với số chữ số thập phân cố định chính là gợi ý làm điều đó.

2. Bao lồi của tôi sai khi có nhiều điểm thẳng hàng. Xử lý thế nào?

| Vấn đề nằm ở chỗ bạn dùng dấu “lớn hơn” hay “lớn hơn hoặc bằng” khi loại điểm khỏi ngăn xếp, và câu trả lời phụ thuộc vào việc đề muốn gì: giữ mọi điểm trên cạnh của bao, hay chỉ giữ các đỉnh thật sự. Hãy đọc kỹ đề, chọn một quy ước, và kiểm bằng một ví dụ có ba điểm thẳng hàng trên biên. Ngoài ra nhớ loại các điểm trùng nhau trước khi chạy, vì chúng gây ra tích có hướng bằng 0 ở những chỗ không mong muốn.

3. Có nên tự cài các thuật toán hình học phức tạp trong thi đấu không?

| Với bao lồi, giao đoạn thẳng, diện tích, sắp xếp theo góc thì có — chúng ngắn và bạn nên có sẵn trong thư viện riêng. Với tam giác phân Delaunay, giao nửa mặt phẳng, hay bao lồi ba chiều thì hãy cân nhắc: chúng dài, nhiều trường hợp suy biến, và thường có cách vòng qua. Trước khi cài một thuật toán hình học nặng, hãy dành năm phút hỏi “bài này có cách nào không cần hình học không” — nhiều bài trông hình học lại giải được bằng sắp xếp cộng một quan sát.

4. Làm sao kiểm thử một chương trình hình học?

| Ba lớp dữ liệu, và lớp thứ ba là quan trọng nhất. Một: dữ liệu ngẫu nhiên toạ độ lớn — bắt lỗi tràn. Hai: dữ liệu ngẫu nhiên toạ độ *nhỏ*, ví dụ trong $[-3, 3]$ — điều này tạo ra rất nhiều điểm thẳng hàng và trùng nhau, tức là ép trường hợp suy biến xuất hiện. Ba: dữ liệu dựng tay cho từng trường hợp suy biến bạn nghĩ ra được. Lớp thứ hai là mẹo hiệu quả nhất và bị bỏ qua nhiều nhất — toạ độ nhỏ khiến suy biến xảy ra liên tục thay vì hiếm.

5. Vì sao đề bài thi thường cho toạ độ nguyên?

| Chính vì lý do ở mục 6: với toạ độ nguyên, mọi vị từ tính được chính xác tuyệt đối, nên bài toán có một đáp án xác định và việc chấm không phụ thuộc vào cách cài số thực của thí sinh. Nếu bạn thấy đề cho toạ độ thực với số chữ số thập phân giới hạn, gần như chắc chắn ý định là bạn nhân lên thành số nguyên. Đây là một gợi ý đọc đề mà nhiều người bỏ qua.

6. Cây k -d và bao lồi khác nhau về vai trò thế nào?

| Bao lồi là một *kết quả* — nó mô tả hình dạng bao ngoài của tập điểm. Cây k -d là một *cấu trúc tra cứu* — nó tổ chức tập điểm để trả lời nhiều truy vấn về lân cận. Chúng trả lời hai loại câu hỏi khác nhau: “hình bao của tập này là gì” so với “điểm nào gần điểm truy vấn nhất”. Trong ứng dụng robot, bao lồi hay dùng để mô tả vật cản, còn cây k -d dùng để khớp đám mây điểm.

7. Xoay hệ toạ độ một góc ngẫu nhiên có thực sự giúp gì không?

| Có, và nó là một mẹo thực dụng đáng biết. Rất nhiều trường hợp suy biến trong quét đường liên quan tới các đoạn thẳng đứng hoặc các điểm cùng hoành độ. Xoay toàn bộ tập điểm một góc ngẫu nhiên nhỏ làm xác suất còn tồn tại các trường hợp đó gần bằng 0, nên bạn không phải viết mã xử lý riêng cho chúng. Cái giá: bạn buộc phải chuyển sang số thực, nên đây là đánh đổi giữa hai loại rủi ro — và nó chỉ đáng khi mã xử lý suy biến phức tạp hơn nhiều so với rủi ro sai số.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Vị từ định hướng và kiểm tra hai đoạn thẳng cắt nhau	CSES “Line Segment Intersection”[2]	Xử lý đủ trường hợp thẳng hàng chồng nhau; kiểm bằng toạ độ nhỏ.
● Diện tích đa giác bằng công thức dây giày	CSES “Polygon Area”[2]	Giữ nguyên số nguyên, in ra diện tích nhân đôi để tránh số thực.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Điểm nằm trong, trên biên hay ngoài đa giác	CSES “Point in Polygon”[2]	Bắn tia; xử lý trường hợp điểm nằm đúng trên biên trước.
•• Bao lồi	CSES “Convex Hull”[2]	Quét đơn điệu; quyết định rõ quy ước về điểm thẳng hàng trên biên.
•• Cặp điểm gần nhất	CSES “Minimum Euclidean Distance”[2]	Chia để trị ($C/14$) hoặc quét đường với tập có thứ tự; cài cả hai nếu có thời gian.
•• Số điểm nguyên trên biên và bên trong đa giác	CSES “Polygon Lattice Points”[2]	Định lý Pick cộng gcd cho số điểm trên cạnh — một liên hệ đẹp với $D/21$.
••• Tìm mọi giao điểm của n đoạn thẳng	Bài kinh điển	Quét đường với tập có thứ tự; đây là bài kiểm tra khả năng xử lý suy biến của bạn.
••• Hợp diện tích của n hình chữ nhật	Bài kinh điển	Quét đường cộng cây phân đoạn ($B/10$); một trong những bài ghép hai chương đẹp nhất.
••• Cặp điểm xa nhất trong tập	Bài kinh điển	Bao lồi rồi hai con trỏ quay quanh bao; chứng minh vì sao đáp án luôn nằm trên bao.
••• Hình chữ nhật nhỏ nhất bao quanh một tập điểm	Bài kinh điển	Một cạnh của hình chữ nhật tối ưu luôn trùng một cạnh của bao lồi — hãy tự chứng minh trước.
••• Kiểm thử bao lồi bằng dữ liệu toạ độ nhỏ	Bài tập kiểm thử	Sinh ngẫu nhiên toạ độ trong $[-3, 3]$ và so với vết cạn; mẹo hiệu quả nhất của chương.

Kết chương và đọc tiếp

Hình học là miền mà thuật toán đơn giản còn cài đặt thì không, và lý do nằm ở chỗ số học được dùng để ra quyết định rời rạc. Ba điều đáng mang theo: vị từ định hướng là viên gạch của mọi thứ; số nguyên là bạn còn số thực là nguồn rủi ro; và mẹo kiểm thử bằng toạ độ nhỏ để ép trường hợp suy biến xuất hiện.

Hai chương đầu của phần D là hai miền chuyên. Chương tiếp theo là chương quan trọng nhất của cả phần, và nó thay đổi cách bạn phân bổ thời gian khi gặp bài lạ: cách nhận ra một bài toán *không* có lời giải nhanh nào được biết tới, vì sao thông tin đó đáng giá, và bốn cách sống chung với nó ($D/23$).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates”	Discrete & Computational Geometry, 1997	Nguồn của vị từ thích nghi. Đọc phần mở đầu và phần chặn sai số; mã nguồn kèm theo dùng được trực tiếp.
nền tảng	de Berg, Cheong, van Kreveld, Overmars, <i>Computational Geometry: Algorithms and Applications</i>	Springer, tái bản 2008	Sách trực chính của chương. Đọc chương bao lồi và chương quét đường; phần suy biến nằm rải rác trong các mục “degeneracies”.
tra khi cần	Gajentaan & Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry”	Computational Geometry: Theory and Applications, 1995	Vì sao nhiều bài hình học phẳng đứng yên ở n^2 : chúng khó ngang bài 3SUM ($D/24$).
tra khi cần	Knuth, <i>The Art of Computer Programming</i> , tập 1	Addison-Wesley, tái bản 1997	Tra khi cần cơ sở về số học máy tính và biểu diễn dấu phẩy động.

Nguồn

- [1] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, vol. 18, pp. 305–363, 1997.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).